CENTRO UNIVERSITÁRIO MAX PLANCK (UNIMAX)

CIÊNCIA DA COMPUTAÇÃO

GUILHERME DE FARIA GIMENES

DESENVOLVIMENTO DE UM PRODUTO DE SOFTWARE COM INTELIGÊNCIA ARTIFICIAL E "VIBE CODING": ANÁLISE DOS BENEFÍCIOS, LIMITAÇÕES E PONTOS CRÍTICOS

INDAIATUBA

2026

GUILHERME DE FARIA GIMENES

DESENVOLVIMENTO DE UM PRODUTO DE SOFTWARE COM INTELIGÊNCIA ARTIFICIAL E "VIBE CODING": ANÁLISE DOS BENEFÍCIOS, LIMITAÇÕES E PONTOS CRÍTICOS

Trabalho de Conclusão de Curso apresentado ao curso de Ciência da Computação do Centro Universitário Max Planck (UniMAX), como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Luiz Claudio Chiavini Oliveira Junior.

INDAIATUBA

2026

SUMÁRIO

1 INTRODUÇÃO 6

1.1 Contextualização e Problema de Pesquisa 6

1.2 Objetivos e Justificativa 6

2 FUNDAMENTAÇÃO TEÓRICA 8

2.1 Vibe Coding: Origem e Conceito 8

2.2 Modelos de Linguagem de Grande Escala (LLMs) na Geração de Código 8

2.3 Gamificação no Contexto Acadêmico 9

3 DESENVOLVIMENTO 10

3.1 Engenharia de Contexto e Definição de Personas para as IAs 10

3.2 Metodologia de Validação de Código Gerado 12

3.3 Análise Comparativa (Especialista vs. Leigo) 13

4 CONSIDERAÇÕES FINAIS 14

REFERÊNCIAS 15

DESENVOLVIMENTO DE UM PRODUTO DE SOFTWARE COM INTELIGÊNCIA ARTIFICIAL E "VIBE CODING": ANÁLISE DOS BENEFÍCIOS, LIMITAÇÕES E PONTOS CRÍTICOS

Resumo: A emergência das Inteligências Artificiais (IAs) Generativas e de Modelos de Linguagem de Grande Escala (LLMs) introduziu novos paradigmas na engenharia de software, popularizando práticas como o *vibe coding* (desenvolvimento orientado a linguagem natural). Este trabalho tem como objetivo investigar empiricamente até que ponto o uso de IAs colaborativas pode acelerar o ciclo de vida de um software sem comprometer sua qualidade, segurança e escalabilidade. A metodologia consistiu no desenvolvimento de um Produto Mínimo Viável (MVP), um aplicativo gamificado para os alunos da rede UniEduk, utilizando modelos de linguagem de grande escala das principais empresas do setor (Anthropic, OpenAI, xAI e Google), atuando como co-desenvolvedores desde a ideação até a codificação. Como etapa preliminar, foram construídos arquivos de contexto (personas) específicos para cada IA, padronizando o conhecimento do projeto fornecido a cada modelo. Para a validação, foi realizada uma análise comparativa observando a interação de um usuário sem conhecimento técnico em contraste com um engenheiro de software pleno. Por se tratar de uma entrega parcial, os resultados são apresentados de forma qualitativa. As observações preliminares indicam que a qualidade do software gerado tende a ser maior quando há direcionamento técnico mais preciso, que problemas e falhas de segurança são identificados com mais facilidade nessas condições e que a tendência das IAs à alucinação é mais bem controlada quando o usuário compreende, em nível técnico, aquilo que está solicitando. Conclui-se que a IA atua como um poderoso acelerador técnico, porém exige validação humana contínua e arquitetura pré-definida para evitar a degradação e inconsistência do produto final.

Palavras-chave: Engenharia de Software; Inteligência Artificial; Modelos de Linguagem; Gamificação.

Abstract: The emergence of Generative Artificial Intelligence (AI) and Large-Scale Language Models (LLMs) has introduced new paradigms in software engineering, popularizing practices such as vibe coding (natural language-driven development). This work aims to empirically investigate the extent to which the use of collaborative AI can accelerate the software lifecycle without compromising its quality, security, and scalability. The methodology consisted of developing a Minimum Viable Product (MVP), a gamified application for students of the UniEduk network, using large language models from the leading companies in the sector (Anthropic, OpenAI, xAI, and Google), acting as co-developers from ideation to coding. As a preliminary stage, context files (personas) were built for each AI, standardizing the project knowledge provided to each model. For validation, a comparative analysis was performed observing the interaction of a user without technical knowledge in contrast to a fully qualified software engineer. As this is a partial submission, the results are presented qualitatively. Preliminary observations indicate that the quality of the generated software tends to be higher when there is more precise technical guidance, that problems and security flaws are more easily identified under these conditions, and that the AIs' tendency to hallucinate is better controlled when the user technically understands what is being requested. It is concluded that AI acts as a powerful technical accelerator, but requires continuous human validation and a predefined architecture to avoid degradation and inconsistency in the final product.

Key-words: Software Engineering; Artificial Intelligence; Language Models; Gamification.

1 INTRODUÇÃO

No contexto atual da engenharia de software, a dependência crescente de ferramentas generativas levanta questões críticas sobre a manutenibilidade e a capacidade das IAs em lidar com cenários complexos reais. A prática apelidada de *vibe coding*, onde especificações lógicas dão lugar a descrições em linguagem natural iterativas, altera o fluxo de trabalho do desenvolvedor (ZHAO et al., 2023). Essa mudança desloca o esforço cognitivo do programador da escrita linha a linha para a especificação de intenções e a curadoria das saídas do modelo, o que tem implicações diretas sobre produtividade, qualidade e segurança do produto final.

1.1 Contextualização e Problema de Pesquisa

Muitos iniciantes lançam produtos com falhas de segurança ou arquitetura devido à confiança cega nos *outputs* das IAs, enquanto profissionais experientes perdem tempo revalidando códigos. O problema central da pesquisa baseia-se na ausência de análises empíricas sobre o uso colaborativo de múltiplas LLMs em todo o ciclo de desenvolvimento, sintetizado na seguinte questão: até que ponto a utilização de IA pode acelerar o processo sem comprometer as boas práticas de engenharia de software?

1.2 Objetivos e Justificativa

O objetivo geral deste trabalho é investigar empiricamente até que ponto o uso colaborativo de IAs generativas pode acelerar o ciclo de desenvolvimento de um produto de software completo sem comprometer sua qualidade, segurança e escalabilidade. Para alcançar este objetivo, foram definidos os seguintes objetivos específicos:

a) Analisar comparativamente a qualidade, consistência e limitações das respostas geradas por modelos de linguagem de empresas distintas (Anthropic, OpenAI, xAI e Google) em diferentes etapas do desenvolvimento de software, como design, modelagem de banco de dados e codificação;

b) Desenvolver um Produto Mínimo Viável (MVP) de aplicativo gamificado para a rede UniEduk, utilizando IAs como co-desenvolvedoras em todas as etapas, da prototipação ao código, documentando decisões, acertos e falhas ao longo do processo;

c) Avaliar o impacto da experiência técnica do usuário nos resultados obtidos com o vibe coding, comparando a interação de um engenheiro de software sênior com a de um usuário sem formação técnica, identificando boas práticas, pontos críticos e riscos de cada abordagem;

d) Identificar e documentar os principais pontos críticos do uso de IAs generativas no desenvolvimento de software, especialmente no que diz respeito a alucinações, inconsistências entre modelos, limitações de contexto e geração de código com falhas de segurança, propondo diretrizes práticas para mitigação desses riscos.

A justificativa deste estudo reside na lacuna entre a rápida adoção do vibe coding no mercado e a escassez de evidências empíricas que orientem seu uso responsável. Ao documentar o desenvolvimento real de um produto com quatro famílias de modelos, o trabalho oferece subsídios práticos tanto para iniciantes quanto para profissionais que pretendem integrar IAs generativas a seus fluxos de trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Vibe Coding: Origem e Conceito

O termo *vibe coding* foi popularizado por Andrej Karpathy, pesquisador e cofundador da OpenAI, em 2025, para descrever uma prática de desenvolvimento na qual o programador abdica parcialmente do controle direto sobre o código e passa a colaborar com modelos de linguagem por meio de descrições em linguagem natural (KARPATHY, 2025). Nessa abordagem, o desenvolvedor descreve a intenção do sistema e não a implementação, e a IA gera, revisa e refatora o código de forma iterativa. O resultado é uma alteração substancial no papel do engenheiro: de escritor de código para revisor e curador de código gerado por máquina.

Do ponto de vista da Engenharia de Software, essa prática desafia princípios consolidados por autores como Pressman e Maxim (2016), que enfatizam o planejamento rigoroso, a definição de requisitos e a rastreabilidade das decisões de arquitetura. O *vibe coding*, por natureza, é incremental e conversacional, o que pode resultar em inconsistências arquiteturais, dívida técnica acumulada e dificuldade de manutenção, especialmente em projetos de maior escala (SOMMERVILLE, 2018).

2.2 Modelos de Linguagem de Grande Escala (LLMs) na Geração de Código

Os Modelos de Linguagem de Grande Escala (LLMs) são sistemas de inteligência artificial treinados em vastos corpora textuais, capazes de gerar, completar e transformar texto com alta fluidez, incluindo código-fonte. Zhao et al. (2023) destacam que, embora esses modelos demonstrem desempenho notável em diversas tarefas, eles apresentam limitações significativas: tendência à alucinação (geração de conteúdo plausível, porém incorreto), ausência de memória de longo prazo entre sessões e dificuldade em manter coerência em contextos extensos. No domínio específico da geração de código, Chen et al. (2021), ao avaliarem o modelo Codex sobre o conjunto HumanEval, demonstraram que mesmo modelos de alto desempenho enfrentam dificuldades com cadeias longas de operações e com a vinculação correta de operações a variáveis, o que reforça a necessidade de validação sistemática do código produzido.

No ecossistema atual, as principais empresas do setor (Anthropic, OpenAI, xAI e Google) lançam novos modelos e versões com frequência, cada um com

características distintas de raciocínio, janela de contexto e especialização em código. Essa dinâmica acelerada torna impraticável fixar a análise em versões específicas de modelos; por isso, este trabalho opta por avaliar as famílias de modelos de cada empresa, considerando as características geracionais e arquiteturais que definem cada plataforma, e não versões pontuais sujeitas a atualização contínua.

2.3 Gamificação no Contexto Acadêmico

A gamificação é definida como a aplicação de elementos e mecânicas típicas de jogos, como pontos, níveis, ranking e recompensas, em contextos não lúdicos, com o objetivo de aumentar o engajamento e a motivação dos usuários (DETERDING et al., 2011). No ambiente acadêmico, essa estratégia tem sido utilizada para promover participação ativa, recompensar bom desempenho e criar senso de pertinência e competição saudável entre estudantes. O aplicativo desenvolvido neste trabalho incorpora esses princípios por meio de um sistema de *tiers* (Bronze, Prata, Ouro e Diamante), medalhas, rankings e mural de conquistas, servindo simultaneamente como estudo de caso para a avaliação das IAs na construção de *features* sociais e de gamificação.

3 DESENVOLVIMENTO

Para testar os modelos das empresas selecionadas (Anthropic, OpenAI, xAI e Google), o desenvolvimento foi dividido em camadas lógicas: Autenticação, Banco de Dados, Gamificação e Frontend. Antes de iniciar a codificação, contudo, foi necessário estabelecer um alicerce de contexto compartilhado entre todas as IAs, descrito na seção a seguir.

3.1 Engenharia de Contexto e Definição de Personas para as IAs

Como os LLMs não possuem memória persistente entre sessões e cada família de modelos reage de maneira distinta às instruções recebidas, a primeira etapa prática do desenvolvimento consistiu na *engenharia de contexto*: a construção de arquivos de contexto reutilizáveis, aqui denominados *personas*, que padronizam o conhecimento do projeto fornecido a cada IA no início de cada interação. O objetivo dessa padronização foi reduzir a variabilidade das respostas e garantir que os quatro modelos partissem das mesmas premissas arquiteturais, permitindo uma comparação mais justa entre eles.

Cada arquivo de persona foi estruturado em duas camadas. A primeira, comum a todos os modelos, descrevia o núcleo invariável do projeto: o nome e o propósito do aplicativo gamificado da rede UniEduk, a pilha tecnológica adotada, as convenções de código, o modelo de dados e as regras de negócio da gamificação (limiares de pontuação para cada tier, lógica de medalhas e rankings). A segunda camada, específica de cada plataforma, atribuía um papel (persona) ao modelo e ajustava o formato das instruções às características conhecidas de cada família, explorando seus pontos fortes.

O conteúdo comum dos arquivos de contexto foi organizado nos seguintes blocos:

a) Identidade do projeto: nome, objetivo pedagógico, público-alvo (alunos da rede UniEduk) e escopo do MVP;

b) Pilha tecnológica e arquitetura: linguagens, frameworks de frontend e backend, serviço de autenticação e banco de dados relacional, além das fronteiras entre as camadas;

c) Convenções de código: padrões de nomenclatura, estilo, organização de pastas e exigência de testes unitários automatizados para toda lógica de negócio;

d) Regras de gamificação: faixas de pontuação dos tiers Bronze, Prata, Ouro e Diamante, regras de progressão e critérios de concessão de medalhas;

e) Restrições de segurança: tratamento de dados sensíveis, validação de entradas e proibição de expor segredos ou credenciais no código gerado.

A camada específica de cada modelo definiu a persona e o mecanismo de contexto mais adequado a cada plataforma. Para a Anthropic (família Claude), foi utilizado um arquivo de instruções de projeto detalhado, atribuindo ao modelo o papel de “arquiteto de software sênior”, com ênfase em rigor arquitetural e justificativa explícita das decisões, abordagem que se beneficia da boa aderência do modelo a instruções estruturadas e extensas. Para a OpenAI (família GPT), empregou-se um conjunto de instruções personalizadas configurando a persona de “desenvolvedor full-stack pragmático”, focado em entregar implementações funcionais e idiomáticas. Para a xAI (família Grok), a persona foi configurada por meio de instruções de sistema mais concisas e conversacionais, privilegiando respostas diretas. Para o Google (família Gemini), aproveitou-se a ampla janela de contexto para ingerir arquivos inteiros do projeto, configurando a persona de “revisor de código e documentação”, responsável por avaliar consistência entre módulos.

O Quadro 1 sintetiza a configuração das personas adotadas para cada família de modelos.

Quadro 1: Configuração das personas por família de modelos

Empresa / Família	Mecanismo de contexto	Persona atribuída	Ponto forte explorado
Anthropic (Claude)	Arquivo de instruções de projeto detalhado	Arquiteto de software sênior	Aderência a instruções extensas e rigor arquitetural
OpenAI (GPT)	Instruções personalizadas	Desenvolvedor full-stack pragmático	Implementações funcionais e idiomáticas
xAI (Grok)	Instruções de sistema concisas	Programador objetivo	Respostas diretas e velocidade
Google (Gemini)	Ingestão de arquivos via ampla janela de contexto	Revisor de código e documentação	Análise de consistência entre módulos

Fonte: elaborado pelo autor (2026).

A adoção dessas personas trouxe ganhos imediatos de consistência: ao reiniciar uma sessão, bastava fornecer novamente o arquivo de contexto

correspondente para que o modelo retomasse as premissas do projeto, mitigando parcialmente a limitação de memória de longo prazo apontada por Zhao et al. (2023). Por outro lado, observou-se que a eficácia das personas depende da disciplina do desenvolvedor em mantê-las atualizadas: quando o arquivo de contexto divergia do estado real do código, os modelos passavam a gerar sugestões incoerentes, evidenciando que a engenharia de contexto é, ela própria, uma fonte potencial de dívida técnica.

3.2 Metodologia de Validação de Código Gerado

Para garantir que a “aceleração” não comprometeu a “qualidade”, o código gerado pelas IAs foi submetido a testes unitários rigorosos. Foi avaliado se a IA conseguiu, por exemplo, gerar a lógica de subida de *tiers* (Bronze, Prata, Ouro, Diamante) sem erros de lógica de fronteira (*boundary conditions*).

Apoio Técnico ao Aluno: como as IAs podem gerar códigos que parecem corretos mas falham em produção (alucinações lógicas), o aluno deve implementar testes unitários automatizados para validar as respostas das LLMs. Abaixo, um framework de exemplo usando *Jest* (JavaScript) para testar o sistema de pontuação gamificada que a IA criará:

```
// Exemplo de Teste Unitário para validar o código gerado pela IA (Gamificação)
// Função hipotética gerada pela IA que calcula o Tier do aluno
const { calcularTierAluno } = require('./gamificacaoService');

describe('Validação de Lógica da IA - Sistema de Tiers UniEduk', () => {
  test('Deve retornar Bronze para pontuação menor que 1000', () => {
    expect(calcularTierAluno(500)).toBe('Bronze');
  });

  test('Deve retornar Prata para pontuação entre 1000 e 2999', () => {
    expect(calcularTierAluno(1000)).toBe('Prata');
    expect(calcularTierAluno(2999)).toBe('Prata');
  });

  test('Deve tratar dados inválidos (null, strings) por erro de tipagem', () => {
    expect(() => calcularTierAluno(null)).toThrow('Pontuação inválida');
    expect(() => calcularTierAluno('mil pontos')).toThrow('Pontuação inválida');
  });
});
```

A suíte de testes acima exemplifica a estratégia adotada para toda lógica crítica: cada função gerada por uma IA só era incorporada ao MVP após passar por casos de teste que verificavam condições de fronteira, entradas inválidas e comportamento esperado. Esse procedimento converteu a validação subjetiva (“o

código parece correto”) em uma verificação objetiva e repetível, tornando comparáveis os resultados produzidos pelas diferentes famílias de modelos.

3.3 Análise Comparativa (Especialista vs. Leigo)

A análise comparativa foi conduzida observando duas formas de interação com as mesmas IAs e os mesmos arquivos de persona: a de um engenheiro de software pleno e a de um usuário sem formação técnica. O engenheiro, ao revisar criticamente cada resposta, identificava alucinações lógicas, apontava falhas de segurança e refinava os prompts com vocabulário técnico preciso, obtendo código mais robusto a cada iteração. O usuário leigo, por sua vez, tendia a aceitar as primeiras respostas como definitivas, reproduzindo no produto eventuais falhas arquiteturais e de segurança não percebidas.

Essa diferença de postura evidenciou que o fator determinante para a qualidade do resultado não é apenas o modelo escolhido, mas a capacidade do usuário de avaliar criticamente as saídas. As observações preliminares apontam que a qualidade do software gerado é maior quando há direcionamento técnico mais preciso, uma vez que o detalhamento do pedido reduz a ambiguidade e orienta o modelo para a solução esperada. Nessas condições, problemas e falhas de segurança também se tornam mais fáceis de identificar, pois o usuário sabe o que esperar de cada trecho de código e consegue reconhecer desvios. Além disso, a tendência das IAs à alucinação mostra-se mais bem controlada quando o usuário compreende, em nível técnico, aquilo que está solicitando, conseguindo formular restrições claras e detectar respostas plausíveis, porém incorretas. Por se tratar de uma entrega parcial, esses achados são apresentados de forma qualitativa, sem consolidação de métricas quantitativas, que serão aprofundadas na versão final do trabalho.

4 CONSIDERAÇÕES FINAIS

Os resultados parciais indicam que o *vibe coding* se mostrou viável principalmente para a geração de componentes isolados e de código repetitivo (boilerplate), nos quais a aceleração é mais evidente e os riscos são mais facilmente contidos por testes automatizados. Para a manutenção da arquitetura de um projeto inteiro, contudo, a prática revelou limitações importantes, sobretudo a janela de contexto finita das IAs, que dificulta a preservação da coerência arquitetural ao longo de muitas sessões, limitação apenas parcialmente mitigada pelos arquivos de persona descritos neste trabalho.

Conclui-se, em caráter preliminar, que a IA atua como um poderoso acelerador técnico, mas exige validação humana contínua e arquitetura pré-definida para evitar a degradação e a inconsistência do produto final. A engenharia de contexto, materializada nos arquivos de persona, mostrou-se uma prática necessária para tornar o uso de múltiplas LLMs comparável e reprodutível, ainda que demande disciplina de manutenção.

Como trabalhos futuros, sugere-se a integração de agentes de IA autônomos dentro de pipelines de integração e entrega contínuas (CI/CD), capazes de executar e validar testes automaticamente a cada sugestão gerada, bem como a investigação de mecanismos de memória externa (como bases de conhecimento vetoriais) para superar a limitação da janela de contexto observada ao longo deste estudo.

REFERÊNCIAS

CHEN, Mark et al. **Evaluating Large Language Models Trained on Code**. arXiv, 2021. Disponível em: https://arxiv.org/abs/2107.03374. Acesso em: jun. 2026.

DETERDING, S. et al. From game design elements to gamefulness: defining “gamification”. In: **Proceedings of the 15th International Academic MindTrek Conference**. New York: ACM, 2011. p. 9-15.

KARPATHY, A. **There's a new kind of coding I call “vibe coding”**. Postagem em rede social (X). 2 fev. 2025. Disponível em: https://x.com/karpathy/status/1886192184808149383. Acesso em: jun. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

ZHAO, Wayne Xin et al. **A Survey of Large Language Models**. arXiv, 2023. Disponível em: https://arxiv.org/abs/2303.18223. Acesso em: jun. 2026.